

New-Horizons Report

Nicon-David Milandru

March 1, 2026

Contents

1	Introduction	1
2	Logical Design	1
2.1	Entities	1
2.2	Relationships	1
3	Relational Design	2
4	Implementation	2
4.1	Data Definition Language	2
4.2	Sample Data	3
4.3	Queries	3
5	Reflection	3

1 Introduction

Travel agencies face complexity in managing **packages**, **bookings**, and **client information** due to multiple interrelated entities.

This project presents the design and implementation of a relational **database system** for a travel agency. The purpose of the system is to manage the entire lifecycle of **clients**, **bookings**, **invoices**, and the structure of **travel packages**. The scope of the project covers conceptual modeling, relational schema generation, sample data population, and validation queries.

The **database** was modeled using **Oracle SQL Developer Data Modeler**, implemented in **SQL**, and tested via **Supabase**. Limitations include the absence of a front-end application and performance optimization for very large datasets.

2 Logical Design

2.1 Entities

The **entity-relationship diagram (ERD)** was created using **Oracle SQL Data Modeler** and includes relevant **entities** for the project. Each **entity** contains well-defined **attributes** with appropriate **data types**, which may be optional or mandatory, and may also serve as **primary keys** or **foreign keys**. The **schema** is fully normalized. Many-to-many relationships between **entities** were resolved through the introduction of **associative entities**.

2.2 Relationships

The **Client** entity has a mandatory one-to-many relationship with the **Booking** entity (a client must have one or more **bookings**).

The **Booking** entity has a mandatory many-to-one relationship with the **Client** entity (each **booking** is made by exactly one **client**). It also has a mandatory one-to-many relationship with the **Invoice** entity (each **booking** is associated with one or more **invoices**). Finally, it has a mandatory many-to-one relationship with the **Travel Package** entity (each **booking** contains exactly one **travel package**).

The **Invoice** entity has a mandatory many-to-one relationship with the **Booking** entity (each **invoice** is associated with exactly one **booking**).

The **Travel Package** entity has an optional one-to-many relationship with the **Booking** entity (a **travel package** may appear in zero or more **bookings**). It also has one-to-many relationships with the **associative entities** **Accommodation Package**, **Destination Package**, **Transport Package**, and **Group Package**, which resolve potential many-to-many relationships. The relationships with **Accommodation Package**, **Destination Package**, and **Transport Package** are mandatory (each **travel package** must include at least one **destination**, **accommodation**, and **transport**). However, the relationship with **Group Package** is optional, as participation in a **group activity** is not required.

The **Destination Package** entity has a mandatory many-to-one relationship with the **Travel Package** entity and a mandatory many-to-one relationship with the **Destination** entity. The **Destination** entity has an optional one-to-many relationship with the **Destination Package** entity.

The same logic applies to the **Accommodation Package** and **Transport Package** entities. Each of these **associative entities** has a mandatory many-to-one relationship with the **Travel Package** entity and a mandatory many-to-one relationship with its respective **entity** (**Accommodation** or **Transport**). Similarly, the **Accommodation** and **Transport** entities have optional one-to-many relationships with their corresponding **associative entities**.

These relationships, together with the introduction of **associative entities**, resolve the potential many-to-many relationships, which are not considered good practice in a **logical relational model**.

The **Group Package** entity has a mandatory many-to-one relationship with the **Travel Package** entity and an optional one-to-many relationship with the **Guide Package** entity. This entity introduces details about **group activities** and also an **associative entity** to represent the assignment of a **guide** to a **group activity**.

The **Guide Package** entity serves as the **associative entity** between **Group Package** and **Guide**. It has a mandatory many-to-one relationship with the **Group Package** entity and a mandatory many-to-one relationship with the **Guide** entity. The **Guide** entity has an optional one-to-many relationship with the **Guide Package** entity. This structure prevents a direct many-to-many relationship between **guides** and **group activities**.

3 Relational Design

The **relational model** was automatically generated from the **logical model (ERD)** using the **Engineer** function provided by **Oracle SQL Data Modeler**. The resulting **schema** provides additional details about the **database structure**, including **attribute data types**, **primary keys**, and **inherited keys**.

4 Implementation

4.1 Data Definition Language

From the **relational model**, **Oracle SQL Data Modeler** allows the generation of a **Data Definition Language (DDL)** script. This is **SQL** code that creates **tables** for all **entities**, **attributes**, and **relationships** defined in the **schema**. For example, the code generated for the **Client** entity is:

```
1 CREATE TABLE CLIENT (  
2     client_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY ,  
3     first_name VARCHAR(50) NOT NULL ,  
4     last_name VARCHAR(50) NOT NULL ,  
5     email VARCHAR(100) NOT NULL UNIQUE ,  
6     phone_number VARCHAR(20) ,  
7     date_of_birth DATE ,  
8     passport_number VARCHAR(20) ,  
9     registration_date DATE NOT NULL ,  
10    loyalty_points INTEGER NOT NULL  
11 );
```

4.2 Sample Data

After generating the **DDL**, I deployed the **database** on **Supabase** in order to execute my scripts. Creating a **project** and running **SQL scripts** on the platform is straightforward and intuitive. Once the **DDL** was executed, I prepared **sample data** to populate the **tables**. For instance, the sample data for the **Destination** entity is as follows:

```
1 -- DESTINATIONS
2 INSERT INTO DESTINATION (country, city, description, airport_code)
3 VALUES
4 ('France', 'Paris', 'City of Light', 'CDG'),
5 ('Italy', 'Rome', 'The Eternal City', 'FCO'),
6 ('Japan', 'Tokyo', 'Modern metropolis', 'HND');
```

4.3 Queries

With the **sample data** loaded into the **database**, we can now query information and test the **structure**. I created a number of **SQL queries** that retrieve meaningful data from the **tables**. For example, the following query calculates the average **guide rating** by **travel package**:

```
1 SELECT
2     tp.package_name,
3     AVG(g.rating) AS avg_guide_rating
4 FROM travel_package tp
5 JOIN group_package gp ON tp.package_id = gp.package_id
6 JOIN guide_package gpk ON gp.group_package_id = gpk.group_package_id
7 JOIN guide g ON gpk.guide_id = g.guide_id
8 GROUP BY tp.package_name
9 ORDER BY avg_guide_rating DESC;
```

5 Reflection

This is a simple project that also has real-life applications, involving multiple tools, and developing a variety of skills.

Starting from the design of a **logical model**, we must consider how to adapt it to a **real-world scenario**, carefully think about **relationships**, and determine the **information** we want to store in the **database**. Transitioning to a **relational model**, we analyze how the **information** interacts with itself (**relationships**), how to distinguish it (**keys**), and how to store it efficiently (**data types**). Next, we learn to use **tools** that generate **SQL scripts** from a **schema**, which forms the **implementation phase**. Finally, we use the **model** and the entered **data** to retrieve **information** through **queries**.

Ultimately, this project is about **information**: how it flows, what it represents, how it is stored, how it interacts, and how we can extract it. It is essentially the process of deriving **information** from **information**.